claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-ad0eb8d6d42c6a616.jsonl`
- SHA-256: `c9536fdf6ed17ce02e8ba9e81a8fd24e2a3fb92e43668aa0a7f4c49714a33dc3`
- Source modified: `2026-06-10T05:43:05+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T05:41:08.945Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("testing") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"No tests for any FastAPI endpoint (/api/process, /api/query, /api/compile, /api/config) ? no TestClient anywhere","severity":"high","area":"API layer (backend/main.py)","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:91","evidence":"grep for `TestClient|backend.main` across tests/ matches only two docstring mentions (tests/test_config.py:92, tests/test_hashing.py:4). backend/main.py (426 lines) defines /api/process (L91-168: validator gating, default-segmenter fallback via getattr chain, Failure-result handling, was_reingest detection, report emission in a finally block), /api/query (L172), /api/compile (L183-207), /api/config (L87), plus CLI main()/run_cli_diagnose_clip ? none exercised by any test.","why_it_matters":"This is the integration seam where registries, typed config, DB, segmenters, and reporting meet; the compiler SyntaxError proves regressions here ship invisibly. Contract changes (e.g. Result vs list return from segmenters, response shape consumed by the frontend) would not be caught."}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T05:41:16.286Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:64:python -m backend.main --server --port 8000

```
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:72:python -m backend.main --video-
path "C:/path/to/my_video.mp4"
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:78:python -m backend.main --video-
path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:79:python -m backend.main --video-
path "C:/path/to/my_video.mp4" --segmenter gemini
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:80:python -m backend.main --video-
path "C:/path/to/my_video.mp4" --segmenter motion
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:86:python -m backend.main --video-
path "C:/path/to/my_video.mp4" --debug-clip 125.5
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:93:python -m backend.main --video-
path "C:/path/to/my_video.mp4" \
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:183:    python -u -m backend.main
--video-path "C:/path/to/video.mp4"
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:193:python -m backend.main
--video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-frames 300
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:207:   python -m backend.main
--video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:210:   python -m backend.main
--video-path "C:/path/to/video.mp4" --segmenter gemini
C:\Users\avidu\Projects\badminton-highlight-indexer\README.md:213:   python -m backend.main
--video-path "C:/path/to/video.mp4" --segmenter motion
C:\Users\avidu\Projects\badminton-highlight-indexer\TEST_RUN_SUMMARY.md:60:    python -u -m
backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-frames 500
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DEFERRED_HARDENING_PLAN.md:36:- Console
entry points: `indexer = backend.main:main` (so `indexer --server` works),
```

3. user (2026-06-10T05:41:16.951Z)

```
1       import os
2       import sys
3       import argparse
4       import json
5       import logging
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
10
11      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
12      logging.basicConfig(
13          level=logging.INFO,
14          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
15          datefmt='%H:%M:%S'
16      )
17      logger = logging.getLogger(__name__)
18      from fastapi import FastAPI, HTTPException
19      from fastapi.middleware.cors import CORSMiddleware
20      from pydantic import BaseModel
21      from typing import List, Dict, Any, Optional
22
23      from fastapi.staticfiles import StaticFiles
24      from fastapi.responses import HTMLResponse
25      from backend.pipeline.validators.base import registry
26      from backend.infrastructure.database import Database
27      from backend.pipeline.segmenters.base import segmenter_registry, BasePlaySegmenter
28      from backend.pipeline.stitcher.compiler import VideoCompiler
29      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
30      from backend.config import load_config as load_app_config, AppConfig  # new typed config
31      from backend.results import Result, Failure  # standardized error/result contract
32      from backend.reporting import emit_indexer_report
33
34      app = FastAPI(title="Sports Highlight Indexer API")
35
```

```
    # Enable CORS for local web interface
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    # Serves static frontend files directly (now under api/ per approved structure)
    os.makedirs("backend/api/static", exist_ok=True)
    app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")

    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = "backend/api/static/index.html"
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"

    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[AppConfig] = None  # now typed (Pydantic model)

    # Legacy thin wrapper for any remaining direct calls during transition.
    # New code should import load_app_config / AppConfig from backend.config directly.
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str

    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

    @app.post("/api/process")
    def process_video(req: ProcessRequest):
        if not os.path.exists(req.video_path):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")

        video_id = compute_video_id(req.video_path)
        was_reingest = db_instance.get_video(video_id) is not None

```

```
99          # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
100         logger.info(f"Running validations for {req.video_path}...")
101         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
102         failures = [r for r in val_results if not r.passed]
103
104         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
105         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
106         seg_name = req.segmenter or default_seg
107         segmenter_actual = None
108         segmentation_ran = False
109         response: Optional[Dict[str, Any]] = None
110         try:
111             if failures:
112                 db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
113                 response = {
114                     "video_id": video_id,
115                     "status": "failed",
116                     "message": "Video failed validation checks.",
117                     "results": [r.dict() for r in val_results],
118                 }
119                 return response
120
121             # 2. Run segmenter & indexer
122             print(f"[API] Video passed checks. Segmenting and indexing...")
123             db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
124
125             segmenter_cls = segmenter_registry.get(seg_name)
126             if not segmenter_cls:
127                 available = list(segmenter_registry.list_available().keys())
128                 raise HTTPException(
129                     status_code=400,
130                     detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
131                 )
132
133             print(f"[API] Using segmenter: {seg_name}")
134             segmenter_actual = seg_name
135             segmenter = segmenter_cls(db_instance, global_config)
136             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
137             segmentation_ran = True
138
139             if isinstance(result, Failure):
140                 print(f"[API] Segmenter failed: {result.message}")
141                 response = {
142                     "video_id": video_id,
143                     "status": "failed",
144                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
145                     "results": [r.dict() for r in val_results],
146                     "play_segments": [],
147                 }
148                 return response
149
150             segments = result.value if hasattr(result, "value") else result
151
152             response = {
153                 "video_id": video_id,
154                 "status": "success",
155                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
```

```
                "results": [r.dict() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths

    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]

        # Run compiler
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        compiler = VideoCompiler(output_dir)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
            return {"status": "success", "filepath": out_path}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")

    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now
```

```
216
217         # 1. Validation check diagnostics
218         print("[DIAGNOSTIC] Running validation framework...")
219         results = registry.run_all(video_path, cfg)
220         for r in results:
221             status_symbol = "[PASS]" if r.passed else "[FAIL]"
222             print(f"  {status_symbol} {r.validator_name}: {r.message}")
223             if r.details:
224                 print(f"         Details: {r.details}")
225
226         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
227         print("-" * 60)
228         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
229         import cv2
230         cap = cv2.VideoCapture(video_path)
231         if not cap.isOpened():
232             print("[FAIL] OpenCV could not open video.")
233             return
234
235         fps = cap.get(cv2.CAP_PROP_FPS)
236         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
237
238         start_frame = max(0, int((timestamp - 3.0) * fps))
239         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
240
241         # Measure frame-by-frame changes in sample region
242         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
243         prev_gray = None
244         motions = []
245
246         for f in range(start_frame, end_frame):
247             ret, frame = cap.read()
248             if not ret:
249                 break
250             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
251             gray = cv2.GaussianBlur(gray, (21, 21), 0)
252
253             if prev_gray is not None:
254                 diff = cv2.absdiff(prev_gray, gray)
255                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
256                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
257                 motions.append(motion_ratio)
258             prev_gray = gray
259
260         cap.release()
261
262         if motions:
263             avg_motion = sum(motions) / len(motions)
264             # Support both legacy dict (from wrapper) and future direct model
265             if hasattr(cfg, "indexing"):
266                 threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
267             else:
268                 threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
269             print(f"  Sample Frame Count: {len(motions)}")
270             print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
271             if avg_motion > threshold:
272                 print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
273             else:
274                 print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
275         else:
```

```
276             print("  [ERROR] No visual frames were extracted in the sample range.")
277
278         print("=" * 60)
279
280     def main():
281         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
282         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
283         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
284         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
285         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
286         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
287         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
288         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
289
290         available_segmenters = list(segmenter_registry.list_available().keys())
291         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
292         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
293         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
294         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
295
296         args = parser.parse_args()
297
298         if args.setup:
299             # Invoke setup diagnostics
300             import subprocess
301             print("[INFO] Invoking setup.py...")
302             subprocess.run([sys.executable, "setup.py"])
303             return
304
305         if args.trim_start is not None and args.trim_end is not None:
306             if not args.video_path:
307                 print("[ERROR] Please provide --video-path to extract a segment.")
308                 return
309
310             # Determine output path
311             out_filename = args.trim_output or "trimmed_segment.mp4"
312             if os.path.isabs(out_filename):
313                 out_path = out_filename
314                 out_dir = os.path.dirname(out_path)
315                 out_name = os.path.basename(out_path)
316             else:
317                 out_dir = args.output_dir
318                 out_path = os.path.join(out_dir, out_filename)
319                 out_name = out_filename
320
321             os.makedirs(out_dir, exist_ok=True)
322             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
323
324             compiler = VideoCompiler(out_dir)
325             try:
326                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
327                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
```

```
            except Exception as e:
                print(f"[ERROR] Failed to compile trimmed segment: {e}")
            return

        if args.debug_clip is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to debug a specific clip.")
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        global_config.output.output_dir = args.output_dir
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
        db_instance = Database(db_path)

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
            print(f"[CLI] Processing video file: {args.video_path}")
            if not os.path.exists(args.video_path):
                print(f"[FAIL] Video file not found: {args.video_path}")
                return

            video_id = compute_video_id(args.video_path)
            was_reingest = db_instance.get_video(video_id) is not None

            # Validate (with-context variant surfaces ffprobe_meta for the report)
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
            failures = [r for r in val_results if not r.passed]

            # Save video status
            status = "failed" if failures else "processed"
            db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])

            seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
            segmenter_actual = None
            segmentation_ran = False
            try:
                print("\n" + "="*40)
                print("VALIDATION SUMMARY")
                print("="*40)
                for r in val_results:
                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("="*40 + "\n")

                if failures:
                    print("[FAIL] Video failed checks. Indexing halted.")
                    return

                # Index
                segmenter_cls = segmenter_registry.get(seg_name)
```

```
389                 if not segmenter_cls:
390                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
391                     return
392 
393                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
394                 segmenter_actual = seg_name
395                 segmenter = segmenter_cls(db_instance, global_config)
396                 result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
397                 segmentation_ran = True
398 
399                 if isinstance(result, Failure):
400                     print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
401                     print(f"[FAIL] Indexing halted for video.")
402                     segments = []
403                 else:
404                     segments = result.value if hasattr(result, "value") else result
405                     print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
406             finally:
407                 # Always emit the per-video observability report. Never raises.
408                 paths = emit_indexer_report(
409                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
410                     val_results=val_results, val_context=val_context,
411                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
412                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
413                 )
414                 if paths:
415                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
416                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
417                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
418             return
419 
420         # Start Local Server Mode
421         if args.server or not args.video_path:
422             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
423             uvicorn.run(app, host="127.0.0.1", port=args.port)
424 
425     if __name__ == "__main__":
426         main()
427 
```

4. user (2026-06-10T05:41:25.020Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py:91:    /api/compile and
the CLI resolve the same value. Default + file override + the
```

5. user (2026-06-10T05:41:25.481Z)

```
No files found
```

6. user (2026-06-10T05:41:34.677Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\test_thresholds.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_annotations.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_batch.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_classifier.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_metrics.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_training.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_geometry.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_providers.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_sport_profiles.py
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_video.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_annotation_providers.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_calibration.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_calibrate_wasb.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_export_wasb_segments.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_gemini_refine.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_provider_gemini.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_calibrate_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_distill_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\scratch\test_ai_segmenter.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_automated_provider.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_database.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_harness.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_regression.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_gemini.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_hit_detector.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_rally_gate.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_tracknet_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_validators_base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_wasb_cache.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_yolo_hybrid.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_hashing.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_rally_slicer.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_spec_faq.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_tracknet_runner.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_eval_partial_labels.py
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_rally_seq_proto.py
```

7. user (2026-06-10T05:41:36.171Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_base.py:14:    def
process_video(self, video_id, video_path, player_pool=None, max_frames=None) -> Result:
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_base.py:33:    success =
inst.process_video("v1", "path")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_base.py:39:    failure =
inst.process_video("v1", "fail")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py:92:    runtime mutation
that backend/main.py::main performs for --output-dir."""
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_database.py:42:    segments =
db.query_play_segments("vid1", {"server": "PlayerA"})
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_hashing.py:4:(backend/main.py +
the three run_*.py scripts) into one shared function.
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_gemini.py:29:    res =
segmenter.process_video("v1", "dummy.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_gemini.py:68:    res =
segmenter.process_video("v1", "dummy.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py:22:    def
query_play_segments(self, video_id, filters):
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_tracknet_hybrid.py:64:    res =
seg.process_video("v1", "clip.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_tracknet_hybrid.py:83:    assert
seg.process_video("v1", "clip.mp4") == []
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_tracknet_hybrid.py:96:    res =
seg.process_video("v1", "clip.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_tracknet_hybrid.py:111:    assert
seg.process_video("v1", "clip.mp4") == []
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_tracknet_hybrid.py:130:
seg.process_video("v1", "clip.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_tracknet_hybrid.py:154:    res =
seg.process_video("v1", "clip.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_yolo_hybrid.py:82:    res =
```

```
segmenter.process_video("v1", "dummy.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_yolo_hybrid.py:106:    res =
segmenter.process_video("v1", "dummy.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_yolo_hybrid.py:149:    res =
segmenter.process_video("v1", "dummy.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_yolo_hybrid.py:218:    res =
segmenter.process_video("v1", "dummy.mp4")
C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_yolo_hybrid.py:269:
segmenter.process_video("v1", "dummy.mp4")
```

8. user (2026-06-10T05:41:47.402Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:81:- `@backend/main.py` ?
Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from
`backend/api/static`. Logging is initialized with `basicConfig` (INFO, timestamped format);
backend modules use `logger = logging.getLogger(__name__)` (replaces stray prints in hot paths
like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries may still use
`print` for direct output.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:176:[Omitted long matching
line]
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:237:- **Config access:**
`backend/main.py` and all four `run_*.py` load via `backend.config.load_config` (typed
`AppConfig`). The legacy `.get()` dict-compat shim remains for validators/segmenters. Remaining
literal: a defensive `getattr(..., 'motion')` fallback in `main.py` that only fires if
`global_config`/`.indexing` is `None` (the ASGI-import edge case).
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:238:- **Segmenter
default:** single source of truth = model `IndexingConfig.default_segmenter='yolo_hybrid'`;
`config.json` may override at runtime (ships `gemini`). run_*.py read the model value (no
literals); `main.py` keeps the defensive `'motion'` getattr fallback noted above.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:242:- **Reporting never
breaks a run:** `emit_indexer_report` swallows all exceptions (designed for `main.py`
`finally:`) and is emitted even on validation failure. `obsreport` is a SOFT dep ? absent -> no-
op.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:273:| Tweak what the report
records | `@backend/reporting/harvest.py::record_run` + stage builders; emit point is
`@backend/main.py` `finally:` -> `@backend/reporting/__init__.py::emit_indexer_report` |
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:279:| Run a video end-to-
end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\CODE_MAP.md:328:| (no dedicated test) |
individual `pipeline/validators/*` impls, `pipeline/stitcher.compiler`, `main.py`,
`run_process_and_stitch.py`, `pipeline/detectors/wasb_runner`,
`pipeline/segmenters/{wasb_hybrid,motion}`, `eval/audio/e1_probe` (diagnostic) ? ? exercised
only indirectly |
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md:68:- ? **Deliverable 1
(Config)** ? `backend/config/` package (`models.py` + `loader.py`) instead of a single
`config.py`. Typed `AppConfig`, central loader with built-in defaults + file override,
`extra="allow"` for migration. Update `setup.py` + `main.py`. Add tests. Land this before the
layout move.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\PLATFORM_ARCHITECTURE.md:58:migration
ladder, v1), `backend/main.py` (endpoints, sync flow), the registry pattern
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\TRACKNET_WSL_SETUP.md:113:Caveat:
WASB's repo documents dataset *evaluation* (`main.py --config-name=eval
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-
moves\LOW_REGRET_MOVES_PLAN.md:61:??? main.py                    # Thin CLI + FastAPI bootstrap
only
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-
moves\LOW_REGRET_MOVES_PLAN.md:101:- Three divergent default sources (`config.json`, `setup.py`,
`main.py` fallbacks).
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-
moves\LOW_REGRET_MOVES_PLAN.md:129:The models are the single source of truth for defaults.
`setup.py` will call the new save function. `main.py` will use the typed loader early.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\past_projects\low-regret-
moves\LOW_REGRET_MOVES_PLAN.md:134:- Update `main.py` (CLI + API paths) to use typed config
(minimal surface at first).
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-logging-
reporting\README.md:12:- Logging setup in `backend/main.py` and hot paths (motion, wasb_infer,
stitcher, config loader, etc.).
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-logging-
reporting\README.md:15:- Integration in `main.py` process_video (always emit in finally),
hybrids, validators.
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-logging-
reporting\README.md:25:- Code: backend/reporting/, backend/main.py (logging.basicConfig),
pipeline/segmenters/* (logger usage).
```

9. user (2026-06-10T05:41:48.606Z)

```
No matches found
```

10. user (2026-06-10T05:42:02.195Z)

```
31:  - `[project.optional-dependencies] dev` ? `pytest`, coverage tooling.
39:  shadows the build system; update README/`run_all_tests.py`/CI references.
50:- `pip install -e .[dev]` in a clean venv ? `run_all_tests.py` green.
85:- Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
```

11. user (2026-06-10T05:42:03.760Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import ast; ast.parse(open('backend/pipeline/stitcher/compiler.py',
encoding='utf-8').read()); print('compiler.py: parses OK')
~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\ast.py", line 50, in parse
    return compile(source, filename, mode, flags,
                   _feature_version=feature_version, optimize=optimize)
  File "<unknown>", line 100
    logger.warning({segment_label}: start_time is negative ({start:.2f}s). Clamping to 0.)
                                                                    ^
SyntaxError: invalid decimal literal
```

12. user (2026-06-10T05:42:23.503Z)

```
7fbbeb0 Sun Jun 7 14:03:29 2026 +0000 feat(logging): standardize to logging in hot paths +
update docs
docs/next-steps-multisport
```

13. user (2026-06-10T05:42:23.942Z)

```
90                  logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
91                  for idx, (raw_start, raw_end) in enumerate(segments):
92                      # Normalize timestamps to float seconds ? handles float, int,
93                      # plain numeric strings, and colon-separated formats like "MM:SS"
94                      start = self._parse_time(raw_start)
95                      end = self._parse_time(raw_end)
96                      segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
97
98                      # Validate the segment times
99                      if start < 0:
100                         logger.warning({segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.)
101                         start = 0.0
102
103                     if start >= end:
104                         logger.error({segment_label}: start_time ({start:.2f}s) >= end_time
({end:.2f}s). Skipping invalid segment.)
```

```
105                         skipped_segments.append((idx, start, end, "start >= end"))
106                         continue
107
108                     # Check if segment extends beyond video duration
109                     if video_duration > 0:
```

14. user (2026-06-10T05:42:25.008Z)

```
No matches found
```

15. user (2026-06-10T05:42:41.639Z)

```
* docs/next-steps-multisport
  master
  remotes/origin/HEAD -> origin/master
  remotes/origin/claude/docs-handoff-consistency
  remotes/origin/claude/platform-architecture-research
  remotes/origin/docs/next-steps-multisport
  remotes/origin/feat/eval-human-labels
  remotes/origin/feat/owned-model-study-and-prototype
  remotes/origin/master
    logger.warning({segment_label}: start_time is negative ({start:.2f}s). Clamping to 0.)
                                                                    ^
SyntaxError: invalid decimal literal
```

16. user (2026-06-10T05:43:02.128Z)

```
Structured output provided successfully
```

17. assistant (2026-06-10T05:43:05.244Z)

```
Verification complete.
```